



INSIGHTSMASTERY ACADEMY

POWER BI DATA MODELING STUDENT HANDBOOK

END-TO-END EDITION

From a Flat Spreadsheet to a Production Semantic Model
From Absolute Beginner to Advanced

STAR SCHEMA • FACTS • DIMENSIONS • GRAIN • KEYS

Power Query Shaping • Date Table • Relationships • Cardinality • Filter Direction

Role-Playing • Multiple Facts • Many-to-Many Bridges • Calculation Groups

Storage Modes • Composite Models • Row-Level Security • VertiPaq Optimization

Validation • Publishing • End-to-End Project

Professional Training Edition

Designed for Classroom & Self-Learning

www.insightsmastry.in



Contents

1. How to Use This Handbook	4
2. What Is Data Modeling?	5
2.1 Simple Meaning	5
2.2 Why Not One Big Flat Table?	5
2.3 Key Vocabulary	5
3. The Business Case and Source Data	7
3.1 The Business Requirements	7
3.2 The Source Extract	7
3.3 Profiling the Source	8
4. Dimensional Modeling Fundamentals	9
4.1 Fact Tables	9
4.2 Dimension Tables	9
4.3 Grain — the Most Important Decision	9
4.4 Star Schema vs Snowflake	10
4.5 Keys	10
4.6 Special Dimension Patterns	11
5. Designing Our Star Schema	12
5.1 Step 1 — Choose the Business Process	12
5.2 Step 2 — Declare the Grain	12
5.3 Step 3 — Identify the Dimensions	12
5.4 Step 4 — Identify the Facts	12
5.5 The Finished Model	13
5.6 Table Specifications	13
6. Shaping the Tables in Power Query	15
6.1 Query Architecture	15
6.2 DimProduct in M	15
6.3 FactSales in M	16
6.4 FactTarget in M	16
6.5 Power Query Best Practices	16
7. The Date Table	18
7.1 Requirements of a Date Table	18
7.2 Turn Off Auto Date/Time	18
7.3 Options for Building It	18
7.4 DimDate with DAX	18
7.5 Finish the Table	19
8. Relationships	20
8.1 Creating Relationships	20
8.2 Cardinality	20
8.3 Cross Filter Direction	20
8.4 Active and Inactive Relationships	21
8.5 Regular vs Limited Relationships	21
8.6 Referential Integrity and the (Blank) Row	21
8.7 Resolving Ambiguity	22
8.8 DAX Functions for Relationships	22
9. Advanced Relationship Patterns	23



9.1 Role-Playing Dimensions	23
9.2 Multiple Fact Tables at Different Grains	23
9.3 Many-to-Many Dimensions with a Bridge Table	24
9.4 Other Patterns in Brief	25
10. Calculations in the Model	26
10.1 Where Should a Calculation Live?	26
10.2 The Core Measures	26
10.3 Organising Measures	27
10.4 Calculation Groups	27
10.5 Field Parameters	27
11. Making the Model Easy to Use	28
11.1 Model View Tools	28
11.2 A Measure in TMDL	28
12. Storage Modes and Composite Models	30
12.1 The Storage Modes	30
12.2 Composite Models	30
12.3 User-Defined Aggregations	30
12.4 Choosing a Mode	31
13. Row-Level Security	32
13.1 Static RLS	32
13.2 Dynamic RLS	32
13.3 Publishing and Role Mapping	32
13.4 RLS Best Practices	33
14. Performance and Size Optimization	34
14.1 How VertiPaq Stores Data	34
14.2 Data Reduction Techniques	34
14.3 Tools	34
14.4 DirectQuery Performance	35
15. Validating the Model	36
15.1 Test 1 — Totals Reconcile with the Source	36
15.2 Test 2 — Every Dimension Filters Correctly	36
15.3 Test 3 — No Orphan Keys	37
15.4 Test 4 — Keys Are Unique	37
15.5 Validation Checklist	37
16. Publishing and Lifecycle	38
17. Common Mistakes and Troubleshooting	39
18. End-to-End Project — Sales Performance Model	40
19. Practice Exercises	42
20. Answer Key	43
21. Data Modeling Cheat Sheet	44
21.1 Sources and Further Reading	44



1. How to Use This Handbook

This handbook is written as if an experienced teacher is sitting beside you, building a real semantic model with you from the first spreadsheet to the published model. You do not need any database background. Every step is done on one small dataset, so you can check every number.

The end-to-end journey

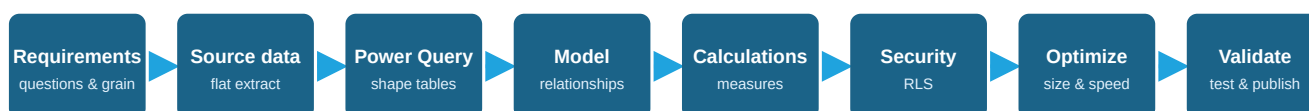


Figure 1.1 — The eight stages this handbook follows, in order

Recommended Learning Path

1. Read Chapters 2–5 slowly. Facts, dimensions and grain are the foundation of everything else.
2. Build the model in Power BI Desktop as you read Chapters 6–9 — Power Query, the date table and relationships.
3. After every chapter, run the validation queries in Chapter 15 and compare with the green **Result** lines.
4. Use Chapters 12–14 (storage modes, security, performance) once your model works.
5. Finish with the End-to-End Project checklist and the practice exercises.

Teaching Pattern Used Throughout

WHAT → WHY → HOW (CLICKS OR CODE) → RESULT → WHY RESULT → REAL-WORLD EXAMPLE → COMMON MISTAKE → PRACTICE

Legend for model diagrams

Symbol	Meaning
Dark blue header	Fact table — events you summarise
Light blue header	Dimension table — things you filter and group by
Brown header	Bridge (factless fact) table — resolves many-to-many
◆ / ◇	Key column (unique) / foreign key column
1 and *	One side and many side of a relationship
Arrow on the line	Direction that filters flow; two arrows = both directions
Dashed line	Inactive relationship



2. What Is Data Modeling?

2.1 Simple Meaning

A **data model** (in Power BI called a **semantic model**) is the set of tables, relationships, calculations and properties that sits between your raw data and your reports. Every visual sends a query to it. A good model answers those queries correctly, quickly and in business language.

Library Analogy

Raw data is a pile of books on the floor. A data model is the library: shelves (tables), a catalogue (relationships), labels in plain language (names and folders), and a librarian who knows the rules (measures). Readers find what they need without knowing where the books came from.

2.2 Why Not One Big Flat Table?

Beginners often load one wide spreadsheet and build reports directly on it. It works for ten rows — and fails for ten million. The table below compares the two approaches.

Problem	One flat table	Star schema
Repeated text	Customer, Product, Category repeated on every row	Stored once in a small dimension
Model size	Large — wide text columns compress poorly	Small — fact table holds narrow numeric keys
Second fact (e.g. targets)	Cannot join cleanly; needs another flat table that cannot filter the first	Shares the same dimensions; one slicer filters both
Time intelligence	No proper date table	Dedicated date dimension
Usability	Dozens of columns in one list	Fields grouped by business entity
Security	Rules on a huge table	Rules on a small dimension that filter the fact

Microsoft's guidance

Dimension tables enable **filtering and grouping**; fact tables enable **summarization**. A well-designed model provides both, and each table should be one or the other — not a mix.

2.3 Key Vocabulary

Term	Meaning	In our model
Semantic model	The Power BI model that reports query (formerly called a dataset)	Sales Performance
Fact table	Stores events or observations with numeric values	FactSales, FactTarget
Dimension table	Describes business entities used to filter and group	DimDate, DimProduct, DimCustomer, DimRegion, DimSalesperson



Term	Meaning	In our model
Grain	What one row of a fact table represents	One order line
Key	Column that uniquely identifies a dimension row	ProductKey
Relationship	A filter path between two tables	DimProduct[ProductKey] 1 → * FactSales[ProductKey]
Measure	A named DAX calculation evaluated at query time	Total Sales
Cardinality	(1) relationship type; (2) number of distinct values in a column	One-to-many; ProductKey has 6 values



3. The Business Case and Source Data

3.1 The Business Requirements

Our company sells electronics and furniture across three regions. Management asks these questions. **Write requirements down before modeling** — they decide the grain, dimensions and measures.

#	Business question	Needs
Q1	What are total sales, cost and profit?	Fact measures
Q2	How do sales split by region, category, product and customer?	Dimensions: Region, Product, Customer
Q3	How do sales trend by day, month and quarter?	Date dimension
Q4	How much did we ship, by ship date?	Second role of the date dimension
Q5	How do actual sales compare with monthly regional targets?	Second fact table at a different grain
Q6	Each salesperson may see only their own sales.	Row-level security

3.2 The Source Extract

IT sends one flat CSV extract, **Sales.csv** — the same 10 rows used in the DAX and Bookmarks handbooks, plus a ship date.

ID	Order Date	Ship Date	Customer	Product	Category	Region	Qty	Unit Price	Sales Amount	Cost	Sales person
1	01-Jan-26	03-Jan-26	Amit	Laptop	Electronics	North	2	50000	100000	80000	Ravi
2	02-Jan-26	04-Jan-26	Neha	Mouse	Electronics	South	5	1000	5000	3000	Priya
3	03-Jan-26	05-Jan-26	Rahul	Keyboard	Electronics	North	3	2000	6000	3500	Ravi
4	04-Jan-26	06-Jan-26	Amit	Monitor	Electronics	West	2	15000	30000	22000	Amit
5	05-Jan-26	07-Jan-26	Priya	Chair	Furniture	South	4	5000	20000	14000	Priya
6	06-Jan-26	08-Jan-26	Rahul	Desk	Furniture	North	1	15000	15000	10000	Ravi
7	07-Jan-26	09-Jan-26	Neha	Laptop	Electronics	South	1	50000	50000	40000	Priya
8	08-Jan-26	10-Jan-26	Amit	Chair	Furniture	West	2	5000	10000	7000	Amit
9	09-Jan-26	11-Jan-26	Rahul	Mouse	Electronics	North	10	1000	10000	6000	Ravi
10	10-Jan-26	12-Jan-26	Priya	Desk	Furniture	South	2	15000	30000	20000	Priya

A second small file, **Targets.xlsx**, holds the monthly target per region:

Month	Region	Target
Jan-2026	North	120,000
Jan-2026	South	110,000
Jan-2026	West	50,000



3.3 Profiling the Source

Before shaping, profile every column (Power Query → View → **Column quality**, **Column distribution**, **Column profile**; set profiling to the entire data set). Note what you find:

Column	Distinct values	Observation	Decision
SalesID	10	Unique per row	Fact key / degenerate dimension
Customer	4	Repeats on many rows	Move to DimCustomer
Product, Category, UnitPrice	6 / 2 / 5	Category and price depend on Product	Move to DimProduct
Region	3	Repeats	Move to DimRegion
Salesperson	3	Repeats	Move to DimSalesperson
Order Date, Ship Date	10 / 10	Two different dates	One DimDate, two roles
Qty, SalesAmount, Cost	numeric	Additive values	Stay in FactSales

Numbers to recognise

Total Sales **276,000** · Total Cost **205,500** · Profit **70,500** · Quantity **32** · North **131,000** · South **105,000** · West **40,000** · Electronics **201,000** · Furniture **75,000** · Target total **280,000**.



4. Dimensional Modeling Fundamentals

4.1 Fact Tables

A fact table stores **events or observations**: sales orders, stock balances, exchange rates, website visits. It contains **dimension key columns** and **numeric columns** that you summarise. Fact tables are usually long and keep growing.

Fact table type	One row represents	Example
Transaction	One event	One order line (our FactSales)
Periodic snapshot	State at a regular point in time	Stock on hand at month end; our monthly FactTarget
Accumulating snapshot	One process with milestone dates that update	Order → shipped → delivered → paid
Factless fact	An event or relationship with no numbers	Salesperson covers Region (bridge, Chapter 9)

4.2 Dimension Tables

A dimension table describes the **things** you model: products, customers, places, people and time. It has one unique key column and descriptive columns used for slicers, rows, columns and filters. Dimensions are usually short and wide.

4.3 Grain — the Most Important Decision

The **grain** is what one fact row represents. Declare it in one sentence before you build anything: *"One row of FactSales is one product on one order."*

- Every fact table must load data at **one consistent grain**.
- The dimension keys define the dimensionality; their values define the granularity. FactTarget has Date and Region keys, but its date values are always the first day of a month — so its grain is **month × region**.
- Choose the lowest grain the questions need. You can always summarise up; you can never drill below the grain.



4.4 Star Schema vs Snowflake

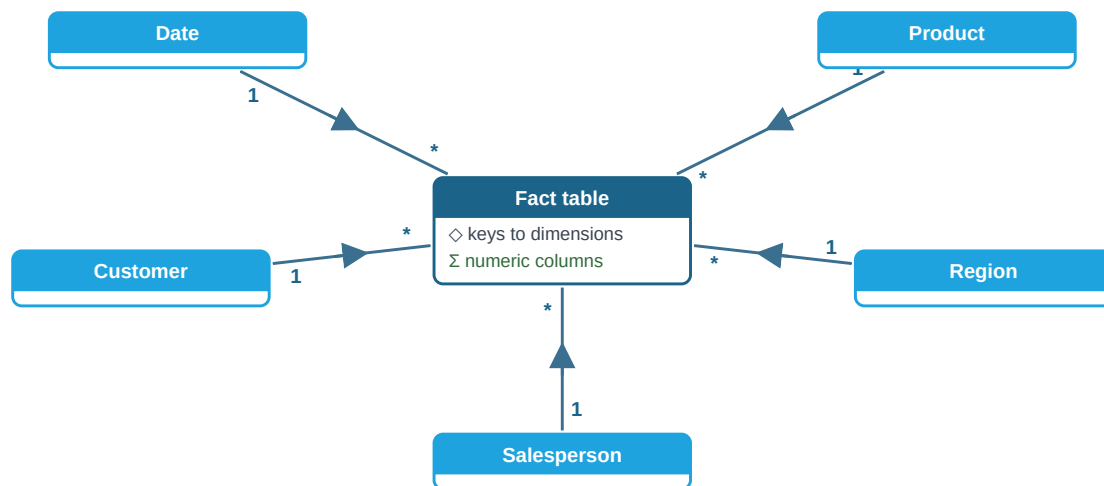


Figure 4.1 — A star schema: one fact table in the centre, dimension tables around it

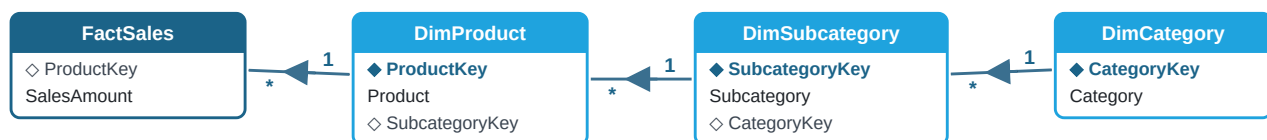


Figure 4.2 — A snowflake: the product dimension split into three normalized tables

Microsoft's guidance: generally the benefits of a single denormalized dimension table outweigh a snowflake. A snowflake loads more tables, creates longer filter chains, clutters the Data pane, and prevents a hierarchy that spans tables (Category → Subcategory → Product needs one table).

Rule of thumb

Flatten each business entity into one dimension table in Power Query. Keep separate tables only for different entities.

4.5 Keys

Key type	Meaning	Example
Natural (business) key	The identifier from the source system	Product name, employee ID
Surrogate key	A unique integer you add for modeling	ProductKey = 1...6 from a Power Query index column
Foreign key	The key column in the fact table that points to a dimension	FactSales[ProductKey]

Power BI relationships join on **one** column. If a dimension has no single unique column, add a surrogate key with **Add Column** → **Index Column**, then merge it into the fact query.



4.6 Special Dimension Patterns

Pattern	What it is	Power BI approach
Role-playing	One dimension used for several meanings (Order Date, Ship Date)	Inactive relationship + USERELATIONSHIP, or a duplicate table per role
Slowly changing — Type 1	Attribute overwritten with the latest value	A normal full refresh achieves this
Slowly changing — Type 2	Versions of a member kept with start/end dates	Build in the warehouse; load the versioned table with a surrogate key
Junk	Many tiny flags combined into one table	Cartesian product of the flags + surrogate key
Degenerate	An identifier kept in the fact table (order number)	Leave it in the fact table
Factless fact / bridge	Keys only, no numbers	Resolves many-to-many between dimensions



5. Designing Our Star Schema

We now apply the four classic design steps to the Sales extract.

5.1 Step 1 — Choose the Business Process

Sales orders (and, separately, sales targets).

5.2 Step 2 — Declare the Grain

Fact table	Grain	Rows
FactSales	One product line on one order	10
FactTarget	One region in one month	3

5.3 Step 3 — Identify the Dimensions

Dimension	Key	Attributes	Rows
DimDate	Date	Year, Quarter, Month, Month No, Year Month, Weekday	365 (full year 2026)
DimCustomer	CustomerKey	Customer, City	4
DimProduct	ProductKey	Product, Category, UnitPrice	6
DimRegion	RegionKey	Region	3
DimSalesperson	SalespersonKey	Salesperson, Email	3

5.4 Step 4 — Identify the Facts

Quantity, SalesAmount and Cost are **additive** — they can be summed across every dimension. Profit is derived (SalesAmount – Cost) and becomes a measure. UnitPrice is **non-additive**: it belongs in DimProduct and must never be summed.



5.5 The Finished Model

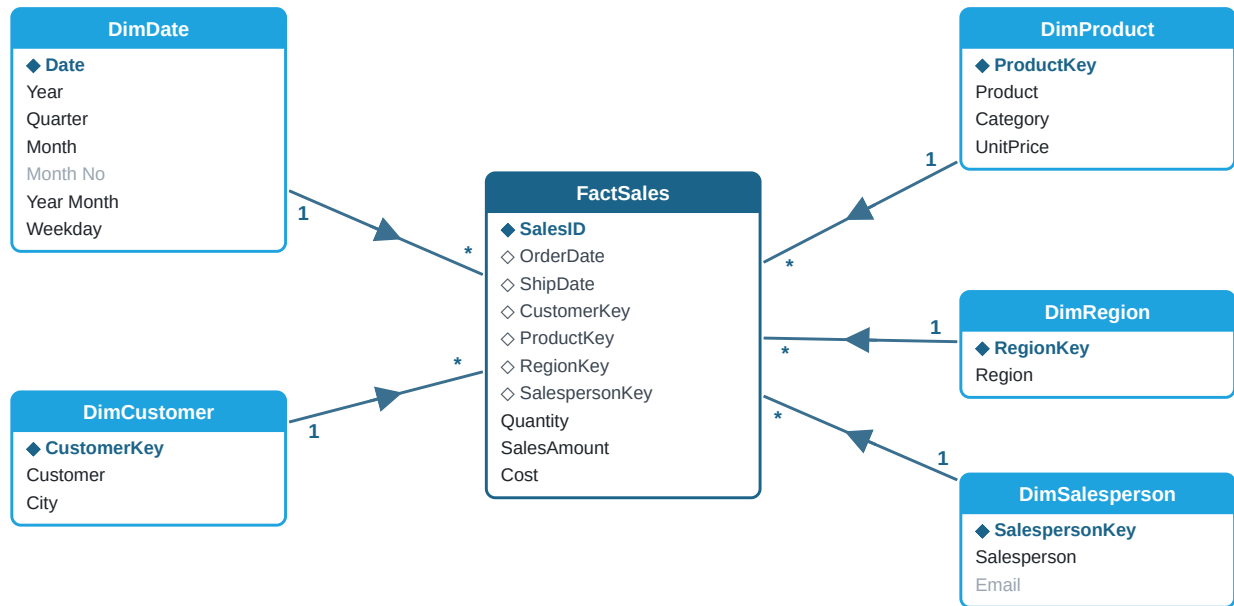


Figure 5.1 — The finished star schema for Sales Performance

5.6 Table Specifications

FactSales

Column	Data type	Role	Visible?
SalesID	Whole number	Degenerate dimension (order line ID)	Yes
OrderDate	Date	FK → DimDate (active)	Hidden
ShipDate	Date	FK → DimDate (inactive)	Hidden
CustomerKey / ProductKey / RegionKey / SalespersonKey	Whole number	FK	Hidden
Quantity	Whole number	Additive fact	Hidden (use measure)
SalesAmount	Fixed decimal	Additive fact	Hidden (use measure)
Cost	Fixed decimal	Additive fact	Hidden (use measure)

Dimension contents

Key	Product	Category	UnitPrice
1	Laptop	Electronics	50,000
2	Mouse	Electronics	1,000



Key	Product	Category	UnitPrice
3	Keyboard	Electronics	2,000
4	Monitor	Electronics	15,000
5	Chair	Furniture	5,000
6	Desk	Furniture	15,000

CustomerKey	Customer	City	RegionKey	Region	SalespersonKey	Salesperson	Email
1	Amit	Delhi	1	North	1	Ravi	ravi@example.com
2	Neha	Mumbai	2	South	2	Priya	priya@example.com
3	Rahul	Kolkata	3	West	3	Amit	amit@example.com
4	Priya	Chennai					

City and Email are illustrative attributes added for this course.

Design check

Every relationship is one-to-many from a dimension to a fact, single direction. No fact relates to another fact. No dimension relates to another dimension. That is a clean star.



6. Shaping the Tables in Power Query

Power Query turns the flat extract into the star schema before data reaches the model. Doing transformations here — as far upstream as possible — gives better compression and a cleaner model than DAX calculated columns.

6.1 Query Architecture



Figure 6.1 — One staging query feeds every dimension and the fact table

1. **Get data** → **Text/CSV** → **Sales.csv** → **Transform data**. Rename the query **Sales_Source**.
2. Set data types on every column (Date, Whole number, Fixed decimal number, Text). Do it once, early.
3. Right-click **Sales_Source** → **Enable load** to turn loading off. It is a staging query only.
4. Right-click **Sales_Source** → **Reference** for each dimension, keep only its columns, **Remove Duplicates**, add an **Index Column** from 1.
5. Reference **Sales_Source** again for **FactSales**, merge each dimension to bring in its key, then remove the text columns.
6. Close & Apply.

6.2 DimProduct in M

```
let
    Source      = Sales_Source,
    Cols        = Table.SelectColumns(Source, {"Product", "Category", "UnitPrice"}),
    Distinct    = Table.Distinct(Cols, {"Product"}),
    Sorted     = Table.Sort(Distinct, {"Product", Order.Ascending}),
    Keyed       = Table.AddIndexColumn(Sorted, "ProductKey", 1, 1, Int64.Type),
    Ordered     = Table.ReorderColumns(Keyed, {"ProductKey", "Product", "Category",
"UnitPrice"})
in
    Ordered
```

Repeat the same five steps for DimCustomer (Customer, City), DimRegion (Region) and DimSalesperson (Salesperson, Email). `Table.Distinct(..., {"Product"})` keeps one row per product even if a price type appears twice.



6.3 FactSales in M

```
let
    Source = Sales_Source,
    MergeP = Table.NestedJoin(Source, {"Product"}, DimProduct, {"Product"}, "P",
JoinKind.LeftOuter),
    ExpP = Table.ExpandTableColumn(MergeP, "P", {"ProductKey"}),
    MergeC = Table.NestedJoin(ExpP, {"Customer"}, DimCustomer, {"Customer"}, "C",
JoinKind.LeftOuter),
    ExpC = Table.ExpandTableColumn(MergeC, "C", {"CustomerKey"}),
    MergeR = Table.NestedJoin(ExpC, {"Region"}, DimRegion, {"Region"}, "R",
JoinKind.LeftOuter),
    ExpR = Table.ExpandTableColumn(MergeR, "R", {"RegionKey"}),
    MergeS = Table.NestedJoin(ExpR, {"Salesperson"}, DimSalesperson, {"Salesperson"},
"S", JoinKind.LeftOuter),
    ExpS = Table.ExpandTableColumn(MergeS, "S", {"SalespersonKey"}),
    Keep = Table.SelectColumns(ExpS, {"SalesID", "OrderDate", "ShipDate",
"CustomerKey",
        "ProductKey", "RegionKey", "SalespersonKey", "Quantity", "SalesAmount",
"Cost"})
in
    Keep
```

Result: FactSales has 10 rows and 10 narrow columns; the six dimension text columns are gone.

6.4 FactTarget in M

```
let
    Source = Excel.Workbook(File.Contents("Targets.xlsx"), true){[Item = "Targets"]}
[Data],
    Typed = Table.TransformColumnTypes(Source, {{"Month", type date}, {"Target",
Currency.Type}}),
    MergeR = Table.NestedJoin(Typed, {"Region"}, DimRegion, {"Region"}, "R",
JoinKind.LeftOuter),
    ExpR = Table.ExpandTableColumn(MergeR, "R", {"RegionKey"}),
    Keep = Table.SelectColumns(ExpR, {"Month", "RegionKey", "Target"})
in
    Keep
```

Month holds the first day of the month (1-Jan-2026). That date relates to DimDate[Date], but the **grain stays month** — Chapter 9 explains how to report it safely.

6.5 Power Query Best Practices

Practice	Why
Remove unneeded columns early	Smaller model; the step can fold back to the source



Practice	Why
Filter rows early (e.g. last 5 years)	Fewer rows to load and refresh
Keep steps foldable for database sources	Query folding pushes work to the source (right-click a step → View Native Query)
Disable load on staging queries	Helper queries should not become model tables
Set data types once, early	Prevents type errors and wrong summarization
Prefer Power Query columns to DAX calculated columns	Better compression and faster refresh
Name steps meaningfully	Easier to maintain and debug



7. The Date Table

A date table is the most common table in any model, and DAX time intelligence needs one. Build it deliberately rather than relying on automatic tables.

7.1 Requirements of a Date Table

1. A column of data type **Date** (or Date/Time) — the date column.
2. Unique values, no BLANKs, no missing dates.
3. Spans **full years** (a fiscal year is fine).
4. Marked as a date table when using classic time intelligence functions.

7.2 Turn Off Auto Date/Time

File → **Options and settings** → **Options** → **Current File** → **Data Load** → **Auto date/time** (and the Global setting for new files). Auto date/time creates a hidden date table for every date column, which inflates the model and cannot filter several fact tables with one date slicer.

7.3 Options for Building It

Technique	When to use
Connect to a warehouse date dimension	Best — one organisation-wide definition of time
Generate in Power Query	No warehouse; you want it computed at refresh
Generate with DAX (CALENDAR / CALENDARAUTO)	Quick and fully under your control
Clone with DAX (Ship Date = 'Date')	A second role-playing date table (columns only; no formats or hierarchies are copied)

7.4 DimDate with DAX

```
DimDate =
VAR StartDate = DATE( 2026, 1, 1 )
VAR EndDate   = DATE( 2026, 12, 31 )
RETURN
    ADDCOLUMNS(
        CALENDAR( StartDate, EndDate ),
        "Year",          YEAR( [Date] ),
        "Quarter",       "Q" & QUARTER( [Date] ),
        "Month No",      MONTH( [Date] ),
        "Month",         FORMAT( [Date], "MMM" ),
        "Year Month",    FORMAT( [Date], "YYYY-MM" ),
        "Month Start",   DATE( YEAR( [Date] ), MONTH( [Date] ), 1 ),
        "Weekday No",    WEEKDAY( [Date], 2 ),
        "Weekday",       FORMAT( [Date], "ddd" )
    )
```



Result: 365 rows, 1-Jan-2026 to 31-Dec-2026.

CALENDARAUTO() instead extends the range automatically to full years covering every date in the model, so it grows at each refresh. Use CALENDAR when you want a fixed, explicit range.

7.5 Finish the Table

1. Select **Month** → Column tools → **Sort by column** → **Month No** (otherwise months sort Apr, Aug, Dec...). Do the same for Weekday → Weekday No.
2. Hide helper columns (Month No, Weekday No).
3. Create a hierarchy **Calendar**: Year → Quarter → Month → Date.
4. Table tools → **Mark as date table** → choose the Date column. Power BI validates uniqueness, no blanks and contiguous dates.
5. Relate DimDate[Date] 1 → * FactSales[OrderDate] (active) and 1 → * FactSales[ShipDate] (inactive).

Calendar-based time intelligence

Microsoft now recommends the newer **calendar-based time intelligence** (in preview at the time of writing) for the best performance and flexibility with any calendar, including fiscal and retail calendars. Classic functions such as DATESYTD and SAMEPERIODLASTYEAR still work and require a marked date table. Microsoft advises against adding many offset columns to the date table just to simplify DAX.



8. Relationships

Relationships are the filter paths of the model. A visual's filters start on dimension tables and travel along relationships to the fact table, where measures add up the remaining rows.

8.1 Creating Relationships

1. **Model view**: drag FactSales[ProductKey] onto DimProduct[ProductKey], or
2. **Modeling** → **Manage relationships** → **New**: choose both tables and columns.
3. Check cardinality, cross filter direction and Active in the dialog before selecting OK.

Power BI auto-detects relationships after loading (File → Options → Data Load → Relationships). Always review what it created — automatic guesses can be wrong when data is incomplete.

8.2 Cardinality

Type	Meaning	Use
One-to-many (1:*) / Many-to-one (*:1)	Unique values on one side, duplicates allowed on the other	Almost every relationship: dimension → fact
One-to-one (1:1)	Unique values on both sides	Rare; often a sign two tables should be merged
Many-to-many (*:*)	Duplicates allowed on both sides	Relating facts at a higher grain, or cross-source; use deliberately

Refresh failure

If a refresh loads a duplicate value into a column on the "one" side, the entire refresh fails. Guarantee uniqueness in Power Query (Remove Duplicates on the key).

8.3 Cross Filter Direction



Figure 8.1 — A Region filter flows to FactSales, but not onwards to DimProduct (single direction)

Cardinality	Allowed directions
One-to-many	Single (from the one side) or Both
One-to-one	Both
Many-to-many	Single either way, or Both

- **Single** — the default and the recommendation for star schemas.



- **Both** (bi-directional) — filters also flow from the many side back to the one side. It can hurt performance, can create ambiguous paths, and Power BI may refuse it. Use only for specific patterns such as a many-to-many bridge.
- Prefer `CROSSFILTER(..., BOTH)` inside one measure over setting a model relationship to Both.

Example: products sold in the North

A slicer on Region = North does **not** reduce the Product slicer — all 6 products still appear, because filters do not flow from FactSales to DimProduct. To show only products with North sales, use the slicer's visual filter **[Total Sales] is not blank**, which is cheaper than making the relationship bi-directional.

8.4 Active and Inactive Relationships

Only one **active** path may exist between two tables. FactSales has two date columns, so DimDate → OrderDate is active and DimDate → ShipDate is **inactive** (dashed line). An inactive relationship is used only inside a calculation:

```
Sales by Ship Date =
CALCULATE(
    [Total Sales],
    USERELATIONSHIP( FactSales[ShipDate], DimDate[Date] )
)
```

Date range (DimDate)	Total Sales (order date)	Sales by Ship Date
1–10 Jan 2026	276,000	236,000
11–12 Jan 2026	(blank)	40,000

8.5 Regular vs Limited Relationships

	Regular	Limited
When	One-to-many or one-to-one within one source group	Many-to-many cardinality, or a cross-source-group relationship in a composite model
Model view sign	Normal line	Parenthesis-like marks after the cardinality indicators
Join behaviour	Table expansion; unmatched fact rows go to a (Blank) member	INNER JOIN; unmatched rows simply disappear
RELATED()	Works	Does not work

8.6 Referential Integrity and the (Blank) Row

If FactSales contained ProductKey 99 with no matching DimProduct row, a regular relationship would show those sales under **(Blank)** product. The total stays correct, but users see an unexplained blank. Fix it upstream: add an **Unknown** member (key -1) to each dimension and map missing keys to it in Power Query.



8.7 Resolving Ambiguity

When bi-directional relationships create several paths, Power BI chooses by priority: paths of one-to-many relationships first, then paths including many-to-many, then many-to-one, and so on; ties are broken by weight, which USERELATIONSHIP can raise. The practical lesson: keep a clean star with single-direction relationships and ambiguity never arises.

8.8 DAX Functions for Relationships

Function	Purpose	Example
RELATED	Fetch a value from the one side (row context)	RELATED(DimProduct[Category])
RELATEDTABLE	Get the related rows on the many side	COUNTROWS(RELATEDTABLE(FactSales))
USERELATIONSHIP	Use an inactive relationship	Sales by Ship Date
CROSSFILTER	Change direction or disable for one calculation	CROSSFILTER(FactSales[ProductKey], DimProduct[ProductKey], BOTH)
TREATAS	Apply values as a filter on an unrelated table (virtual relationship)	TREATAS(VALUES(DimRegion[Region]), Other[Region])
COMBINEVALUES	Build a multi-column key (DirectQuery)	COMBINEVALUES(" ", [A], [B])



9. Advanced Relationship Patterns

9.1 Role-Playing Dimensions

One dimension, several meanings. FactSales has **OrderDate** and **ShipDate**; both relate to dates. There are two designs.

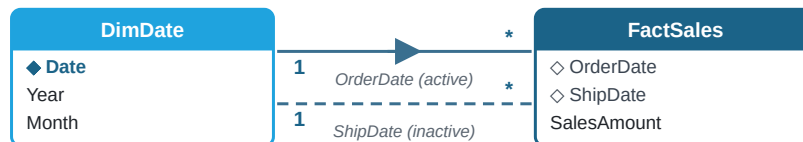


Figure 9.1 — Design A: one date table, active OrderDate path, inactive ShipDate path

	Design A — inactive relationship	Design B — one table per role
Setup	One DimDate; ShipDate relationship inactive	Clone: Ship Date = DimDate, rename its columns (Ship Year, Ship Month...)
Report use	Only through measures with USERELATIONSHIP	Any visual can slice by order date and ship date at the same time
Measures	Extra 'by Ship Date' measures	No extra measures
Best when	Few role-specific measures; visuals never filter both roles together	Users need both roles as slicers, rows or columns

Microsoft generally recommends active relationships where possible, which means duplicating a role-playing dimension (Design B). Use inactive relationships when reports never need to filter by both roles at once.

9.2 Multiple Fact Tables at Different Grains

FactTarget is at **month × region**; FactSales is at order line. Both relate to the shared dimensions DimDate and DimRegion — these are **conformed dimensions**, so one slicer filters both facts.

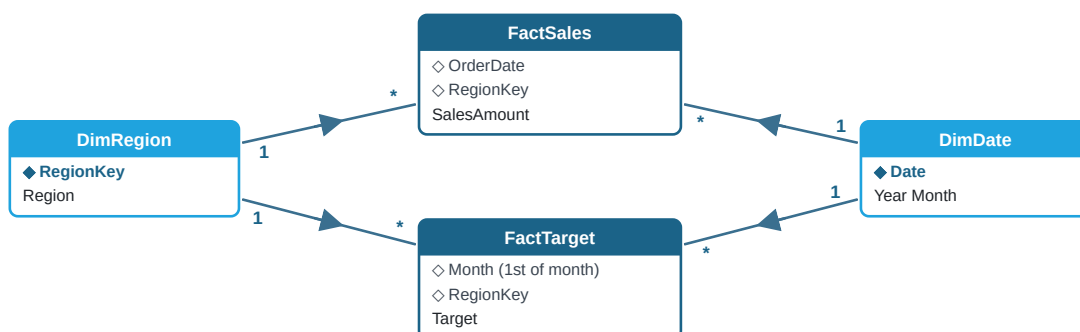


Figure 9.2 — Two facts sharing conformed dimensions; facts never relate to each other

Total Target = SUM(FactTarget[Target])

Achievement % = DIVIDE([Total Sales], [Total Target])



Region	Total Sales	Total Target	Achievement %
North	131,000	120,000	109.17%
South	105,000	110,000	95.45%
West	40,000	50,000	80.00%
Total	276,000	280,000	98.57%



Figure 9.3 — Achievement % by region

Protecting the month grain

All targets sit on 1-Jan-2026. A visual by day would show 280,000 on 1 January and nothing on other days — misleading. Blank the target below month level:

```
Total Target =
IF(
    ISINSCOPE( DimDate[Date] ),
    BLANK(),
    SUM( FactTarget[Target] )
)
```

Microsoft also documents an alternative: relate the higher-grain fact on a month column (e.g. Year Month) with a many-to-many relationship. Whichever you choose, also hide day-level date columns from target-only reports.

9.3 Many-to-Many Dimensions with a Bridge Table

New rule: a salesperson can cover several regions. Priya now covers South **and** West. Store the coverage in a **bridge** (factless fact) table:

SalespersonKey	Salesperson	RegionKey	Region
1	Ravi	1	North
2	Priya	2	South
2	Priya	3	West
3	Amit	3	West



Figure 9.4 — Salesperson filters the bridge; the bridge filters Region only when the measure enables Both; Region filters FactTarget



DimSalesperson also relates directly to FactSales, so making the bridge relationship bi-directional in the model would create two paths from Salesperson to FactSales. Enable it **only inside the measure**:

```
Target by Salesperson =  
CALCULATE(  
    [Total Target],  
    CROSSFILTER( BridgeSalesRegion[RegionKey], DimRegion[RegionKey], BOTH )  
)
```

Salesperson	Regions covered	Target by Salesperson
Ravi	North	120,000
Priya	South, West	160,000
Amit	West	50,000
Total		280,000

Not additive — and that is correct

The rows add to 330,000 but the total shows **280,000**. West's 50,000 target is shared by Priya and Amit, and the total counts it once. Many-to-many measures are **non-additive**; explain this to report users.

9.4 Other Patterns in Brief

Pattern	Guidance
One-to-one	Usually merge the two tables in Power Query. Keep separate only when they come from different sources or one is optional.
Header / detail facts	Order header (freight, order date) and order lines: push header values down to the line grain (allocate freight), or keep the header as a degenerate dimension.
Parent-child (employee → manager)	Flatten with DAX PATH and PATHITEM into fixed level columns, then build a hierarchy.
Fact-to-fact	Never relate two fact tables directly; relate both to shared dimensions.
Dimension-to-dimension	Avoid; flatten the attributes into one dimension (no snowflake).



10. Calculations in the Model

10.1 Where Should a Calculation Live?

Option	Computed	Stored	Use for
Source / SQL view	Before load	Yes	Business rules shared by every tool
Power Query column	At refresh	Yes, well compressed	Row-level attributes (Profit per line, Age band)
DAX calculated column	After load, at refresh	Yes, less efficient	Logic that needs DAX or other tables (PATH, ranking at load)
DAX measure	At query time	No	Every aggregate: totals, ratios, time intelligence, KPIs
Visual calculation	On the visual's result	No	Running sums and comparisons inside one visual

Rule

Shape data as far upstream as possible, and as far downstream as necessary. Columns go upstream (Power Query or the source); aggregations become measures.

10.2 The Core Measures

```

Total Sales      = SUM( FactSales[SalesAmount] )
Total Cost       = SUM( FactSales[Cost] )
Total Profit     = [Total Sales] - [Total Cost]
Profit Margin %  = DIVIDE( [Total Profit], [Total Sales] )
Total Quantity   = SUM( FactSales[Quantity] )
Order Lines      = COUNTROWS( FactSales )
Customers        = DISTINCTCOUNT( FactSales[CustomerKey] )
Avg Selling Price = DIVIDE( [Total Sales], [Total Quantity] )
Revenue at List  = SUMX( FactSales, FactSales[Quantity] * RELATED( DimProduct[UnitPrice] ) )
  
```

Measure	Result	Measure	Result
Total Sales	276,000	Total Quantity	32
Total Cost	205,500	Order Lines	10
Total Profit	70,500	Customers	4
Profit Margin %	25.54%	Avg Selling Price	8,625
Revenue at List	276,000	Total Target	280,000



Revenue at List uses RELATED to read UnitPrice from DimProduct inside the row context of SUMX — possible only because the relationship is regular (one-to-many).

10.3 Organising Measures

- **Explicit measures only** — hide numeric fact columns so users cannot drag them in and get implicit sums. Turn on **Discourage implicit measures** in the model properties.
- **Measure table** — Home → Enter data, one column, name it **_Measures**; move measures there (set each measure's Home table); hide the column. It appears at the top of the Data pane.
- **Display folders** — Model view → Properties → Display folder, e.g. **Sales**, **Profit**, **Targets**. Use \ for subfolders.
- **Format strings** — set once on the measure (**#,##0; 0.00%**). Dynamic format strings can change format by context.
- **Descriptions** — fill in every measure's description; it shows as a tooltip in the Data pane and helps Copilot and Q&A.

10.4 Calculation Groups

A calculation group applies one expression pattern to **any** measure, replacing dozens of near-duplicate measures. Create it in Model view (**Calculation group** on the ribbon) or in TMDL view. **SELECTEDMEASURE()** is the placeholder for whichever measure is in the visual.

```
Calculation group: Time Calc    (column: Time Calc)
```

```
Current = SELECTEDMEASURE()  
MTD      = CALCULATE( SELECTEDMEASURE(), DATESMTD( DimDate[Date] ) )  
YTD      = CALCULATE( SELECTEDMEASURE(), DATESYTD( DimDate[Date] ) )  
PY       = CALCULATE( SELECTEDMEASURE(), SAMEPERIODLASTYEAR( DimDate[Date] ) )
```

Put **Time Calc** on the columns of a matrix with Total Sales and Total Profit in values: four versions of each measure without writing eight measures. On 10 Jan 2026, YTD Sales = 276,000 and PY = blank (no 2025 data). Creating a calculation group turns on Discourage implicit measures automatically. When several calculation groups apply, **precedence** decides the order.

10.5 Field Parameters

Modeling → **New parameter** → **Fields** creates a table that lets users switch which measure or column a visual uses (Sales / Profit / Quantity; Region / Category). It is a model object, so plan its name and place like any table.



11. Making the Model Easy to Use

A correct model can still be hard to use. These properties turn it into a self-service product.

Property / feature	Where	Do this
Names	Rename tables and columns	Business names, spaces allowed, no codes (Sales Amount, not SLS_AMT)
Hide keys	Right-click → Hide in report view	Hide every key and foreign key column
Hide raw facts	Same	Hide numeric fact columns once measures exist
Default summarization	Column tools → Summarization	Don't summarize for Year, UnitPrice, IDs
Sort by column	Column tools	Month by Month No; Weekday by Weekday No
Data category	Column tools	City = City, Country = Country (maps); URL = Web URL / Image URL
Hierarchies	Right-click column → Create hierarchy	Calendar: Year → Quarter → Month → Date; Product: Category → Product
Display folders	Properties pane	Group measures and columns logically
Descriptions	Properties pane	Every table, column and measure that is visible
Synonyms	Properties pane / Q&A setup	Revenue = Sales; Rep = Salesperson

11.1 Model View Tools

- **Layouts** — create a diagram tab per subject (e.g. Sales, Targets, Security) instead of one crowded diagram.
- **Model explorer** — Data pane → Model: a tree of tables, relationships, roles, calculation groups, perspectives and more, with counts.
- **Multi-select** — Ctrl + click several tables or columns and change a property (Is hidden, Storage mode) for all at once.
- **DAX query view** — write and run DAX queries against the model for testing (Chapter 15).
- **TMDL view** — script any object as Tabular Model Definition Language, edit it as code and apply. Generally available in Power BI Desktop.

11.2 A Measure in TMDL

```
createOrReplace

table _Measures

    measure 'Total Sales' = SUM( FactSales[SalesAmount] )
        formatString: #,##0
        displayFolder: Sales
        description: "Sum of order-line sales amount, before tax."
```



Drag a table or measure from the Data pane into TMDL view to script it. Scripts can be saved and reused — for example a standard date table or time-intelligence calculation group shared by the whole team.

Teacher Tip

Many teams keep Dim/Fact prefixes in Power Query but give model tables business names (Sales, Product, Date). This handbook keeps the prefixes so students always see which table is which.



12. Storage Modes and Composite Models

12.1 The Storage Modes

Mode	Where data lives	Strengths	Watch out
Import	Compressed in memory (VertiPaq)	Fastest queries; full DAX and Power Query	Data is as fresh as the last refresh; model size limits
DirectQuery	Stays in the source; queried live	Near real-time; very large data	Slower; source must be tuned; 1,000,000-row intermediate result limit; some modeling limits
Dual	Both imported and DirectQuery	Dimensions answer slicers from memory and join natively to DirectQuery facts	Needs refresh like Import
Direct Lake (Fabric)	Delta tables in OneLake, loaded on demand	Import-like speed without copying data	Fabric capacity; specific limitations
Hybrid table	Import partitions + one DirectQuery partition	Fast history plus live recent data	Set up via incremental refresh; no aggregations on hybrid tables

Default choice

Start with **Import**. Move to DirectQuery, Dual or Direct Lake only for a specific reason: data volume, freshness, or a governed Fabric lakehouse.

12.2 Composite Models

A composite model mixes storage modes or sources. Recommended pattern: large fact tables in **DirectQuery**, their dimensions in **Dual**, small reference tables in **Import**.

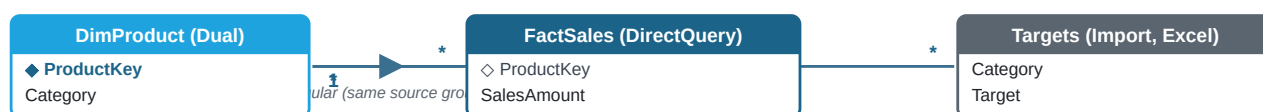


Figure 12.1 — Relationships inside one source group are regular; cross-source relationships are limited

- Relationships across source groups are created as many-to-many and are always **limited**: INNER JOIN behaviour, no RELATED, possible performance cost.
- Use **Dual** for dimensions queried with DirectQuery facts from the same source — it avoids limited relationships.
- Strive for data integrity across sources: limited relationships drop rows whose keys do not match.
- Direct Lake + Import composite models (Fabric web modeling) support **regular** relationships.

12.3 User-Defined Aggregations

An aggregation table (for example sales by Month × Category, imported) answers high-level visuals from memory while detailed visuals still hit the DirectQuery fact. Microsoft's rule of thumb: the aggregation should



have at least **10× fewer rows** than the table it summarises.

12.4 Choosing a Mode

Situation	Choose
Up to tens of millions of rows; daily refresh is fine	Import
Billions of rows or minute-level freshness from a tuned warehouse	DirectQuery facts + Dual dimensions (+ aggregations)
Data already in a Fabric lakehouse or warehouse	Direct Lake
Years of history plus today's transactions	Hybrid table via incremental refresh with real-time data
Extend a published semantic model with your own Excel targets	Composite model (DirectQuery to the semantic model + Import)



13. Row-Level Security

Row-level security (RLS) restricts which rows each viewer sees. Roles hold DAX filter rules; each rule is evaluated per row and the row is visible when it returns TRUE.

13.1 Static RLS

1. **Modeling** → **Manage roles** → **New**; name the role **North Team**.
2. Select table **DimRegion**; rule `[Region] = "North"`.
3. Save. **Modeling** → **View as** → **North Team** to test.

Result: Total Sales = 131,000 for every visual.

13.2 Dynamic RLS

One role filters differently for every user by comparing an email column with the signed-in identity.



Figure 13.1 — The rule filters the small dimension; the relationship carries the filter to the fact

Role: Salesperson
Table: DimSalesperson
Rule: `[Email] = USERPRINCIPALNAME()`

Signed in as	Sees	Total Sales
ravi@example.com	North orders 1, 3, 6, 9	131,000
priya@example.com	South orders 2, 5, 7, 10	105,000
amit@example.com	West orders 4, 8	40,000
anyone else	Nothing	(blank)

Test in Desktop with **View as** → **Other user** plus the role, using the user principal name (UPN). In Desktop, `USERNAME()` returns `DOMAIN\user`; in the service both functions return the UPN.

13.3 Publishing and Role Mapping

- Users are assigned to roles in the **Power BI service** (semantic model → Security), not in Desktop.
- Map roles to **security groups** rather than individuals.
- RLS applies to workspace **Viewers**. Admins, Members and Contributors can edit the model, so RLS does not apply to them.
- Test in the service with **Test as role**; for dynamic RLS and B2B guests, sign in as the real user to be sure.



13.4 RLS Best Practices

- Put rules on **dimension** tables, not fact tables, and let relationships propagate them.
- RLS propagates only through **active** relationships.
- Avoid LOOKUPVALUE in rules when a relationship can do the job.
- Write rules so unexpected values return **no rows**, never all rows.
- Measure the cost with Performance Analyzer: compare visual durations with and without View as.

Safe rule pattern (Microsoft)

A rule like `IF(USERNAME() = "Worker", [Type] = "Internal", TRUE())` shows **everything** to a mistyped value. Test each expected value explicitly and end with FALSE():

```
IF(USERNAME()="Worker", [Type]="Internal", IF(USERNAME()="Manager", TRUE(), FALSE()))
```



14. Performance and Size Optimization

14.1 How VertiPaq Stores Data

Import models are stored column by column and compressed — roughly 10× compression in memory is typical. Each column is encoded separately:

- **Value encoding** for numbers — very efficient.
- **Hash (dictionary) encoding** for text and other non-numeric data — each distinct value gets an ID, so size grows with the number of distinct values.
- Therefore **column cardinality** (number of distinct values) is the main driver of model size, more than row count.

14.2 Data Reduction Techniques

Technique	Example	Effect
Remove unnecessary columns	Drop comments, audit columns, unused IDs	Smaller and faster
Remove unnecessary rows	Load only the last 5 years; parameterised filter	Fewer rows, faster refresh
Group by and summarise	Load daily totals instead of order lines	Big reduction, but less detail
Optimise data types	SO123456 → 123456 as whole number	Value instead of hash encoding
Split date/time	Separate Date and Time columns	Two low-cardinality columns instead of one huge one
Round decimals	Prices to 2 decimal places	Fewer distinct values
Prefer Power Query columns	Profit per line in M, not a DAX calculated column	Better compression, faster refresh
Disable load of staging queries	Sales_Source not loaded	No duplicate tables
Disable auto date/time	Use one DimDate	Removes hidden date tables
Keep relationships single-direction	Use CROSSFILTER in measures if needed	Simpler, faster query plans

14.3 Tools

Tool	Use it to
Performance Analyzer (Desktop, Optimize tab)	Time each visual: DAX query, visual display, other; copy the query
DAX query view (Desktop)	Run and tune DAX queries against the model
View as roles	Measure the cost of RLS
VertiPaq Analyzer in DAX Studio (community tool)	See table and column sizes and cardinality
Best Practice Analyzer in Tabular Editor (community tool)	Scan the model for rule violations (hidden keys, bi-di, formats)



14.4 DirectQuery Performance

- Index the source tables on join and filter columns.
- Keep Power Query steps foldable.
- Use Dual dimensions and aggregations.
- Limit visuals per page and use Apply buttons on slicers.
- Avoid complex DAX that the source cannot translate efficiently.



15. Validating the Model

A model is not finished until its numbers are proven. Use **DAX query view** (left toolbar in Power BI Desktop) to run test queries and compare them with the source.

15.1 Test 1 — Totals Reconcile with the Source

```
EVALUATE
ROW(
    "Sales",    [Total Sales],
    "Cost",     [Total Cost],
    "Profit",   [Total Profit],
    "Quantity", [Total Quantity],
    "Lines",    [Order Lines]
)
```

Sales	Cost	Profit	Quantity	Lines
276,000	205,500	70,500	32	10

15.2 Test 2 — Every Dimension Filters Correctly

```
EVALUATE
SUMMARIZECOLUMNS(
    DimRegion[Region],
    "Sales",    [Total Sales],
    "Target",   [Total Target],
    "Achievement", [Achievement %]
)
```

Region	Sales	Target	Achievement
North	131,000	120,000	109.17%
South	105,000	110,000	95.45%
West	40,000	50,000	80.00%

```
EVALUATE
SUMMARIZECOLUMNS(
    DimProduct[Category],
    "Sales", [Total Sales],
    "Profit", [Total Profit]
)
```



Category	Sales	Profit
Electronics	201,000	46,500
Furniture	75,000	24,000

15.3 Test 3 — No Orphan Keys

```
EVALUATE
FILTER(
    FactSales,
    ISBLANK( RELATED( DimProduct[Product] ) )
)
```

Expected: an empty result. Repeat for every dimension.

15.4 Test 4 — Keys Are Unique

```
EVALUATE
ROW(
    "Product rows", COUNTROWS( DimProduct ),
    "Distinct keys", DISTINCTCOUNT( DimProduct[ProductKey] )
)
```

Expected: 6 and 6. If the numbers differ, the key is not unique and refresh will fail.

15.5 Validation Checklist

- ☐ Grand totals match the source system for each fact.
- ☐ Each dimension filters each fact it should (and no other).
- ☐ No (Blank) members appear in slicers.
- ☐ Role-playing measures (Sales by Ship Date) return 236,000 for 1–10 Jan.
- ☐ Non-additive measures (Target by Salesperson) total 280,000.
- ☐ Every RLS role returns the expected totals.
- ☐ Every visible field has a business name, format and description.



16. Publishing and Lifecycle



Figure 16.1 — From Desktop file to a shared, governed semantic model

Step	What to do
Publish	Home → Publish to a workspace. The semantic model and the report become separate items.
Credentials and gateway	Semantic model settings → Data source credentials; on-premises sources need an on-premises data gateway.
Scheduled refresh	Set the time zone and times. Shared capacity allows up to 8 refreshes a day; Premium / Fabric capacity up to 48.
Incremental refresh	For large facts: define RangeStart/RangeEnd parameters, filter the fact, and set a policy (e.g. store 5 years, refresh last 10 days).
Shared semantic model	Build many thin reports on one model (Get data → Power BI semantic models) instead of copying the model into every report.
Endorsement	Promote, then certify the model so users know it is the trusted source.
Power BI Project (PBIP) + TMDL	Save as .pbip to store the model as text files for Git diffs, code review and branching.
Deployment pipelines	Move content Development → Test → Production with parameter and rule changes.
Documentation	Descriptions in the model, plus a short read-me: grain, sources, refresh, owners.



17. Common Mistakes and Troubleshooting

Symptom	Likely cause	Fix
Same total on every row of a visual	No relationship, or the visual uses a column from a table that cannot filter the fact	Create the relationship; use dimension columns on rows
(Blank) member in a slicer	Fact keys with no match in the dimension	Add an Unknown member; fix keys upstream
Refresh fails: duplicate value on the one side	Dimension key not unique	Remove duplicates on the key in Power Query
Cannot create a relationship — ambiguity	Second path between tables (often bi-directional)	Make it inactive or single-direction; duplicate the dimension
Numbers inflate when two facts are used together	Facts related directly, or fact-to-fact many-to-many	Relate both facts only to shared dimensions
Target appears on 1st of month only	Fact at month grain shown by day	Blank below month (ISINSCOPE) or relate on a month column
Months sort alphabetically	No sort-by column	Sort Month by Month No
Model is huge	High-cardinality text, datetime columns, auto date/time, unused columns	Apply Chapter 14 reduction techniques
Slicer still lists every product	Single-direction filter does not flow from fact to dimension	Visual-level filter [Total Sales] is not blank
RELATED returns an error	Limited relationship (many-to-many or cross-source)	Use a regular relationship or LOOKUPVALUE / TREATAS
RLS shows all data to a user	User is Admin/Member/Contributor, or the rule returns TRUE for unexpected values	Give Viewer role; write fail-safe rules
RLS has no effect on a table	Filter travels only along active relationships	Check relationship paths and directions



18. End-to-End Project — Sales Performance Model

Build the complete model from the two source files. Tick each item.

Requirements

- ☐ Write the six business questions and the grain sentence for each fact.

Power Query

- ☐ Sales_Source staging query with types set and load disabled.
- ☐ DimCustomer, DimProduct, DimRegion, DimSalesperson with index keys.
- ☐ FactSales with keys and numeric columns only (10 rows).
- ☐ FactTarget with RegionKey and first-of-month date (3 rows).

Date

- ☐ Auto date/time off.
- ☐ DimDate for 2026 with sort-by columns, hierarchy, marked as date table.

Relationships

- ☐ Five dimension → FactSales one-to-many, single direction.
- ☐ DimDate → ShipDate inactive.
- ☐ DimRegion and DimDate → FactTarget.
- ☐ (Stretch) BridgeSalesRegion with Target by Salesperson.

Calculations

- ☐ _Measures table with the core measures, formats, folders and descriptions.
- ☐ Sales by Ship Date, Total Target (month-safe), Achievement %.
- ☐ Time Calc calculation group.

Usability

- ☐ Keys and raw facts hidden; Discourage implicit measures on.
- ☐ Product and Calendar hierarchies; data categories set.

Security

- ☐ Dynamic Salesperson role tested for all three emails.

Validation

- ☐ All Chapter 15 tests pass.



Acceptance test

A matrix with Region on rows and Total Sales, Total Target and Achievement % shows North 131,000 / 120,000 / 109.17%, South 105,000 / 110,000 / 95.45%, West 40,000 / 50,000 / 80.00%. Viewed as priya@example.com, only South appears with 105,000.



19. Practice Exercises

1. State the grain of FactSales and FactTarget in one sentence each.
2. Is UnitPrice a fact or a dimension attribute? Why?
3. Which relationship cardinality and direction should DimProduct → FactSales use?
4. Write the Power Query steps to create DimRegion with a surrogate key.
5. Write a DAX date table for 2026 with Year, Month and Month No.
6. Why does a Region slicer not shorten a Product slicer? Give a fix that keeps single direction.
7. Write a measure for sales by ship date. What is the result for 11–12 Jan?
8. Calculate Achievement % for South.
9. Priya covers South and West. What is her target, and why is the total 280,000 not 330,000?
10. Write a dynamic RLS rule. What does Amit see?
11. Name three techniques to reduce model size.
12. When would you choose Dual storage mode?



20. Answer Key

#	Answer
1	FactSales: one product line on one order. FactTarget: one region in one month.
2	A dimension attribute of DimProduct — it describes the product and is non-additive; never sum it.
3	One-to-many from DimProduct to FactSales, single direction, active.
4	Reference Sales_Source → keep Region → Remove Duplicates → Sort → Add Index Column from 1 named RegionKey.
5	ADDCOLUMNS(CALENDAR(DATE(2026,1,1), DATE(2026,12,31)), "Year", YEAR([Date]), "Month", FORMAT([Date], "MMM"), "Month No", MONTH([Date])); sort Month by Month No.
6	Filters flow from the one side to the many side only. Add the visual-level filter [Total Sales] is not blank to the Product slicer.
7	CALCULATE([Total Sales], USERRELATIONSHIP(FactSales[ShipDate], DimDate[Date])) → 40,000 (orders 9 and 10).
8	$105,000 \div 110,000 = 95.45\%$.
9	160,000 (South 110,000 + West 50,000). West's target is shared with Amit; the total counts each region once, so 280,000.
10	[Email] = USERPRINCIPALNAME() on DimSalesperson. Amit sees West orders 4 and 8 — 40,000.
11	Any three: remove columns, remove rows, summarise, optimise data types, split date/time, prefer Power Query columns, disable auto date/time.
12	For dimension tables queried together with DirectQuery fact tables from the same source.



21. Data Modeling Cheat Sheet

Decision	Default answer
Schema	Star: facts in the centre, one flattened table per dimension
Grain	Lowest level the questions need; one grain per fact table
Keys	Unique integer surrogate keys on dimensions; hide all keys
Relationship	One-to-many, dimension → fact, single direction, active
Bi-directional	Avoid in the model; use CROSSFILTER inside a measure
Two dates in a fact	Duplicate the date table, or inactive + USERELATIONSHIP
Second fact	Relate to shared (conformed) dimensions; never fact-to-fact
Many-to-many dimensions	Bridge table; non-additive totals expected
Date table	Auto date/time off; full-year table; mark as date table
New columns	Source or Power Query first; DAX calculated column last
Aggregations	Explicit measures in a measure table; discourage implicit measures
Repeated measure patterns	Calculation group with SELECTEDMEASURE()
Storage mode	Import by default; DirectQuery facts + Dual dimensions for large/real-time
Security	Dynamic RLS on a dimension with USERPRINCIPALNAME(); map to groups
Size	Reduce columns, rows and cardinality
Testing	DAX query view: totals, per-dimension, orphans, unique keys

21.1 Sources and Further Reading

Guidance in this handbook follows Microsoft's official Power BI and Fabric documentation (accessed 1 October 2026):

- Understand star schema and the importance for Power BI — learn.microsoft.com/power-bi/guidance/star-schema
- Model relationships in Power BI Desktop — learn.microsoft.com/power-bi/transform-model/desktop-relationships-understand
- Create and manage relationships — learn.microsoft.com/power-bi/transform-model/desktop-create-and-manage-relationships
- Many-to-many relationships — learn.microsoft.com/power-bi/transform-model/desktop-many-to-many-relationships
- Design guidance for date tables — learn.microsoft.com/power-bi/guidance/model-date-tables
- Time-based calculations — learn.microsoft.com/power-bi/transform-model/desktop-time-intelligence
- Data reduction techniques for Import modeling — learn.microsoft.com/power-bi/guidance/import-modeling-data-reduction
- Composite model guidance — learn.microsoft.com/power-bi/guidance/composite-model-guidance
- Direct Lake overview — learn.microsoft.com/fabric/fundamentals/direct-lake-overview



- Row-level security with Power BI — learn.microsoft.com/fabric/security/service-admin-row-level-security
- RLS guidance — learn.microsoft.com/power-bi/guidance/rls-guidance
- Calculation groups — learn.microsoft.com/power-bi/transform-model/calculation-groups
- Work with TMDL view — learn.microsoft.com/power-bi/transform-model/desktop-tmdl-view
- Create measures (display folders, measure tables) — learn.microsoft.com/power-bi/transform-model/desktop-measures

Continue Your Journey

Next topics: incremental refresh in depth, calendar-based time intelligence, Fabric Direct Lake modeling, deployment pipelines and Git integration, and DAX optimisation with query plans.

Congratulations on completing the Power BI Data Modeling Student Handbook.
You can now design, build, secure and prove a semantic model from end to end.